UTL-X

Format-Agnostic Data Transformation

The Complete Guide

Ir. Marcel A. Grauwen

Creator of UTL-X

Version 1.0 — 2026

Table of Contents

1 Preface ... 14
1.1 Why This Book ... 14
1.2 Who This Book Is For ... 14
1.3 How to Read This Book ... 14
1.4 Conventions Used ... 14
1.5 Online Resources ... 14
2 Introduction ... 16
2.1 The Data Transformation Problem ... 16
2.2 What Is UTL-X? ... 16
2.3 The Swiss Army Knife Analogy ... 16
2.4 UTL-X vs The Alternatives ... 16
2.5 Architecture Overview ... 16
2.6 The UTL-X Ecosystem ... 16
2.7 A First Example ... 16
3 Licensing: Open Source and Commercial ... 17
3.1 UTL-X Is Open Source (AGPL-3.0) ... 17
3.2 What AGPL-3.0 Means in Practice ... 17
3.2.1 You CAN (without paying anything): ... 17
3.2.2 The Key AGPL Condition: ... 17
3.2.3 You CANNOT (without a commercial license): ... 17
3.3 Usage Scenarios and License Requirements ... 17
3.4 Important Clarification: Your .utlx Files Are YOURS ... 17
3.5 The Commercial License Model ... 17
3.5.1 Why a Commercial License Exists ... 17
3.5.2 Commercial License Options ... 17
3.5.3 Dual Licensing Summary ... 17
3.6 Frequently Asked Questions ... 17
3.6.1 “Can I use UTL-X in my company for free?” ... 17
3.6.2 “Do I need to open-source my .utlx files?” ... 17
3.6.3 “Can I use UTL-X in my SaaS product?” ... 17
3.6.4 “What if I deploy UTL-X from Azure Marketplace?” ... 17
3.6.5 “Can I redistribute UTL-X?” ... 17
3.6.6 “Does AGPL affect my entire application?” ... 17
4 Transformation: The Cornerstone of Integration ... 18
4.1 Why Transformation Matters ... 18
4.2 The Three Pillars of Integration ... 18
4.3 Transformation in ESB Architecture ... 18
4.4 Transformation in Workflow/BPM ... 18
4.5 Why UTLXe Needs Messaging Capabilities ... 18
4.6 The Canonical Data Model Pattern ... 18
5 Getting Started ... 19
5.1 Installation ... 19
5.2 Your First Transformation ... 19
5.3 The .utlx File Structure ... 19
5.4 Identity Mode (The Flip) ... 19

5.5 Expression Mode (-e) ... 19
5.6 The CLI Reference ... 19
5.7 VS Code Extension ... 19
5.8 Project Structure ... 19
6 The Three Executables: utlx, utlxd, and utlxe ... 20
6.1 One Language, Three Runtimes ... 20
6.2 utlx — The CLI (Command Line Interface) ... 20
6.3 utlxd — The Daemon (IDE Integration) ... 20
6.4 utlxe — The Engine (Production Runtime) ... 20
6.5 Comparison Table ... 20
6.6 Architecture: Shared Core, Different Shells ... 20
6.7 When to Use Which ... 20
6.8 The Wrapper Pattern ... 20
7 The UTL-X IDE ... 21
7.1 IDE Layout and Panels ... 21
7.2 Design-Time vs Runtime ... 21
7.3 Schema-to-Schema Mapping (Design-Time First) ... 21
7.4 Scaffolding ... 21
7.5 The UDM Tree Browser ... 21
7.6 The Function Library ... 21
7.7 The %utlx Header Editor ... 21
8 Language Fundamentals ... 22
8.1 Scalar Types ... 22
8.2 Objects ... 22
8.3 Arrays ... 22
8.4 The $input Variable ... 22
8.5 Variable Binding (let) ... 22
8.6 Conditional Expressions ... 22
8.7 Comments ... 22
8.8 Type Annotations (Optional) ... 22
9 The Universal Data Model (UDM) ... 23
9.1 Why UDM Exists ... 23
9.2 UDM Types ... 23
9.3 How Formats Map to UDM ... 23
9.4 XML-Specific UDM Conventions ... 23
9.5 UDM Navigation ... 23
9.6 UDM and Type Coercion ... 23
10 Schema-to-Schema Mapping ... 24
10.1 Why Schema Mapping Matters ... 24
10.2 How Traditional Tools Do It (and Why It's Broken) ... 24
10.2.1 The XSLT Approach (Tibco BW, SAP CPI, Azure) ... 24
10.2.2 The JSON Problem: Why JSON→XML→XSLT→XML→JSON Is Wrong ... 24
10.2.3 The UTL-X Approach: Format-Agnostic Schema Mapping ... 24
10.3 Tier 1: Instance Data Mapping ... 24
10.3.1 Every Data Format Has an Accompanying Schema Format ... 24
10.3.2 Instance-to-Instance Mapping Examples ... 24

10.4 Tier 2: Schema-to-Schema (Metadata) Mapping . . . 24
10.4.1 Why Metadata Mapping Is Powerful . . . 24
10.4.2 All Possible Schema-to-Schema Combinations . . . 24
10.4.3 Why This Matters in Practice . . . 24
10.5 JSONATA: JSON Schema to JSON Schema Mapping . . . 24
10.6 Nested Schemas: Include and Import . . . 24
10.7 USDL: The Universal Schema Definition Language . . . 24
11 USDL: Universal Schema Definition Language . . . 25
11.1 Why USDL? . . . 25
11.2 USDL Syntax . . . 25
11.3 USDL Type System . . . 25
11.4 USDL Constraints . . . 25
11.5 USDL ↔ Other Schema Formats . . . 25
11.6 Using USDL in Transformations . . . 25
11.7 USDL as Documentation . . . 25
12 Format Support Overview . . . 26
12.1 Tier 1: Data Formats . . . 26
12.2 Tier 2: Schema Formats . . . 26
12.3 Format Detection . . . 26
12.4 Format Options . . . 26
12.5 Format Conversion Matrix . . . 26
13 Expressions and Operators . . . 28
13.1 Arithmetic Operators . . . 28
13.2 Comparison Operators . . . 28
13.3 Logical Operators . . . 28
13.4 String Operators . . . 28
13.5 Object Construction . . . 28
13.6 Array Construction . . . 28
13.7 Pipe Operator (|>) . . . 28
13.8 Nullish Coalescing and Safe Navigation . . . 28
14 Functions and Lambdas . . . 29
14.1 Lambda Expressions . . . 29
14.2 User-Defined Functions . . . 29
14.3 Higher-Order Functions . . . 29
14.4 Function Composition . . . 29
14.5 Closures . . . 29
14.6 The Standard Library (Overview) . . . 29
15 Standard Library . . . 30
15.1 String Functions (83 functions) . . . 30
15.2 Array Functions (67 functions) . . . 30
15.3 Math Functions (37 functions) . . . 30
15.4 Date/Time Functions (68 functions) . . . 30
15.5 Type Functions (27 functions) . . . 30
15.6 Encoding Functions (30 functions) . . . 30
15.7 Security Functions (16 functions) . . . 30
15.8 XML Functions (60 functions) . . . 30

15.9 Other Categories ... 30
16 Security Library and Message Security ... 31
16.1 The UTL-X Security Library (stdlib-security) ... 31
16.2 Should Messages Be Encrypted in Transit? ... 31
16.2.1 Transport-Level Encryption (mTLS) ... 31
16.2.2 Message-Level Encryption (End-to-End) ... 31
16.2.3 Why Encrypt-Transform-Re-encrypt Is a Maintenance Burden ... 31
16.3 Digital Signatures and Integrity ... 31
16.3.1 JCS (JSON Canonicalization Scheme) ... 31
16.3.2 XML Digital Signatures (XMLDSig) ... 31
16.3.3 JWS (JSON Web Signature) ... 31
16.4 Future Security Functions (Vision) ... 31
16.4.1 Encryption ... 31
16.4.2 Signing ... 31
16.4.3 Key Management Integration ... 31
16.5 Large Files: CMIS and Object Storage References ... 31
16.5.1 The Problem with Large Payloads ... 31
16.5.2 The Claim Check Pattern ... 31
16.5.3 CMIS (Content Management Interoperability Services) ... 31
16.5.4 Avoid Base64 Embedding ... 31
17 Pattern Matching and Control Flow ... 32
17.1 Match Expressions ... 32
17.2 Try/Catch ... 32
17.3 Conditional Patterns ... 32
17.4 Real-World Patterns ... 32
18 Schema Validation ... 33
18.1 Why Validate? ... 33
18.2 Supported Schema Formats ... 33
18.3 Pre-Validation and Post-Validation ... 33
18.4 JSON Schema Validation ... 33
18.5 XSD Validation ... 33
18.6 Schema-Driven Transformations ... 33
19 Pipeline Chaining and Multi-Hop Transformations ... 34
19.1 What Is Pipeline Chaining? ... 34
19.2 Why Pipelines? ... 34
19.3 Single-Hop vs Multi-Hop ... 34
19.3.1 Single-Hop (Simple Transformation) ... 34
19.3.2 Multi-Hop (Pipeline) ... 34
19.4 Multi-Hop with Additional Inputs per Stage ... 34
19.4.1 The Problem ... 34
19.4.2 The Solution: Additional Inputs per Hop ... 34
19.4.3 When Is Multi-Hop with Additional Inputs Handy? ... 34
19.5 Defining Pipelines ... 34
19.5.1 Pipeline via UTLXe HTTP API ... 34
19.5.2 Pipeline via CLI ... 34
19.6 Multi-Input Transformations (Single Step) ... 34

19.7 Pipeline Patterns ... 34
19.7.1 Enrichment Pipeline ... 34
19.7.2 Validation Sandwich ... 34
19.7.3 Fan-Out ... 34
19.7.4 Fan-In (Aggregation) ... 34
19.7.5 Conditional Routing ... 34
19.8 Performance Characteristics ... 34
19.9 Status: Multi-Hop Additional Inputs ... 34
20 XML Transformations ... 36
20.1 XML in UTL-X ... 36
20.2 Reading XML ... 36
20.3 Writing XML ... 36
20.4 XML Attributes in JSON/YAML Output ... 36
20.5 Common XML Patterns ... 36
20.6 XML Encoding ... 36
20.7 XML Namespaces Deep Dive ... 36
21 XML Attribute Handling: Design Decisions ... 37
21.1 The Fundamental Problem ... 37
21.2 How UTL-X Handles Attributes Internally (UDM) ... 37
21.3 The _text Convention ... 37
21.4 The writeAttributes Option ... 37
21.5 What Gets Kept vs Dropped (Default) ... 37
21.6 Why Drop by Default? (DataWeave-Compatible) ... 37
21.7 JSON vs YAML Consistency ... 37
21.8 Industry Conventions Reference ... 37
21.9 FHIR XML: The value Attribute Convention ... 37
22 JSON Transformations ... 38
22.1 JSON in UTL-X ... 38
22.2 Reading JSON ... 38
22.3 Writing JSON ... 38
22.4 JSON-Specific Functions ... 38
22.5 Common JSON Patterns ... 38
22.6 JSON Lines (JSONL) ... 38
23 CSV Transformations ... 39
23.1 CSV in UTL-X ... 39
23.2 CSV Options ... 39
23.3 Reading CSV ... 39
23.4 Writing CSV ... 39
23.5 CSV-Specific Functions ... 39
23.6 Common CSV Patterns ... 39
23.7 Regional Formats ... 39
24 YAML Transformations ... 40
24.1 YAML in UTL-X ... 40
24.2 Reading YAML ... 40
24.3 Writing YAML ... 40
24.4 YAML Output Options ... 40

24.5 YAML-Specific Functions ... 40
24.6 Common YAML Patterns ... 40
24.7 YAML ↔ JSON Conversion ... 40
25 OData Transformations ... 41
25.1 What Is OData? ... 41
25.2 OData in UTL-X ... 41
25.3 Reading OData ... 41
25.4 Writing OData ... 41
25.5 OData-Specific Functions ... 41
25.6 Common OData Patterns ... 41
25.6.1 Dynamics 365 Integration ... 41
25.6.2 SharePoint Integration ... 41
25.6.3 SAP OData Integration ... 41
25.6.4 Power Platform ... 41
25.7 OData Schema (EDMX/CSDL) ... 41
25.8 OData ↔ Other Formats ... 41
26 Schema Formats ... 42
26.1 XSD (XML Schema Definition) ... 42
26.2 JSON Schema ... 42
26.3 Avro Schema ... 42
26.4 Protobuf (proto3) ... 42
26.5 OSCH (OData Schema / EDMX) ... 42
26.6 TSCH (Table Schema) ... 42
26.7 USDL (Universal Schema Definition Language) ... 42
27 XSD Design Patterns Deep Dive ... 43
27.1 The Eight XML Schema Design Patterns ... 43
27.1.1 1. Russian Doll ... 43
27.1.2 2. Salami Slice ... 43
27.1.3 3. Venetian Blind ... 43
27.1.4 4. Garden of Eden ... 43
27.1.5 5. Swiss Army Knife (Hybrid) ... 43
27.1.6 6-8. Chameleon, Abstract, Adapter ... 43
27.2 Pattern Detection in UTL-X ... 43
27.3 Pattern Conversion in UTL-X ... 43
27.4 XSD Patterns in Cross-Format Mapping ... 43
28 Cross-Format Patterns ... 44
28.1 The Power of Format Agnosticism ... 44
28.2 XML ↔ JSON Conversion ... 44
28.3 CSV ↔ JSON Conversion ... 44
28.4 XML → CSV (Flattening) ... 44
28.5 JSON → XML (Structuring) ... 44
28.6 Multi-Format Pipelines ... 44
28.7 Data Normalization Patterns ... 44
29 Enterprise Integration ... 46
29.1 UTL-X in Integration Architecture ... 46
29.2 Use Case: European E-Invoicing (Peppol/UBL) ... 46

29.3 Use Case: Healthcare (HL7 FHIR) . . . 46
29.4 Use Case: Financial Services (SWIFT/ISO 20022) . . . 46
29.5 Use Case: Retail/E-Commerce . . . 46
29.6 Message Broker Integration . . . 46
29.7 Error Handling in Production . . . 46
30 Engine Lifecycle: Design-Time, Init-Time, Runtime . . . 47
30.1 The Three Phases . . . 47
30.1.1 Phase 1: Design-Time . . . 47
30.1.2 Phase 2: Init-Time (Load / Compile) . . . 47
30.1.3 Phase 3: Runtime (Execute) . . . 47
30.2 Phase Diagram . . . 47
30.3 How Strategies Affect Each Phase . . . 47
30.4 Worker Threading Model . . . 47
31 Cloud Deployment . . . 48
31.1 UTLXe: The Production Engine . . . 48
31.2 Azure Deployment . . . 48
31.3 GCP Deployment . . . 48
31.4 AWS Deployment . . . 48
31.5 Docker and Kubernetes . . . 48
31.6 Health and Monitoring . . . 48
31.7 Hot Reload . . . 48
32 SDKs and Wrapper Libraries . . . 49
32.1 The Wrapper Pattern . . . 49
32.2 Available SDKs . . . 49
32.2.1 C# / .NET Wrapper (Built) . . . 49
32.2.2 Go Wrapper (Built) . . . 49
32.2.3 Python Wrapper (Planned) . . . 49
32.2.4 Java / Kotlin Wrapper (Direct) . . . 49
32.2.5 Node.js / TypeScript Wrapper (Planned) . . . 49
32.3 Alternative: HTTP API (No SDK Needed) . . . 49
32.4 Alternative: Dapr Sidecar (Cloud-Native) . . . 49
32.5 SDK Comparison . . . 49
33 Migration Guides . . . 50
33.1 Migrating from XSLT . . . 50
33.2 Migrating from DataWeave . . . 50
33.3 Migrating from jq . . . 50
33.4 Migrating from Custom Code . . . 50
33.5 Migrating from Tibco BW / BWCE . . . 50
34 Performance and Optimization . . . 51
34.1 Execution Strategies . . . 51
34.2 Performance Characteristics . . . 51
34.3 Memory Optimization . . . 51
34.4 GraalVM Native Image . . . 51
34.5 Benchmarking . . . 51
34.6 Production Tuning . . . 51
35 Message Parsing, Memory, and Size Boundaries . . . 52

35.1 How UTL-X Parses Messages . . . 52
35.1.1 DOM-Style Parsing (What UTL-X Uses) . . . 52
35.1.2 SAX-Style Parsing (Not Used by UTL-X) . . . 52
35.1.3 StAX-Style Parsing (Not Used by UTL-X) . . . 52
35.1.4 Summary: Parsing Approaches . . . 52
35.2 Memory Expansion Factor . . . 52
35.2.1 The Problem: Why 100KB of XML Becomes 5-10MB in Memory . . . 52
35.2.2 Expansion Factors by Format . . . 52
35.2.3 Real-World Examples . . . 52
35.3 Practical Message Size Boundaries . . . 52
35.3.1 Sweet Spot: 1 KB - 1 MB . . . 52
35.3.2 Feasible: 1 MB - 10 MB . . . 52
35.3.3 Not Recommended: 10 MB - 50 MB . . . 52
35.3.4 Not Feasible: 50 MB+ . . . 52
35.3.5 Size Boundary Summary . . . 52
35.4 Memory Sizing Guide for Production . . . 52
35.4.1 Formula . . . 52
35.4.2 The 75% Rule . . . 52
35.5 Future: Streaming Support? . . . 52
35.5.1 Could UTL-X Support Streaming? . . . 52
35.5.2 Split-Before-Transform Pattern . . . 52
35.6 Comparison: UTL-X vs Streaming Tools . . . 52
36 Logging, Observability, and Compliance . . . 53
36.1 Logging in UTL-X . . . 53
36.1.1 What UTL-X Logs . . . 53
36.1.2 Log Levels . . . 53
36.1.3 Configuring Logging . . . 53
36.2 When NOT to Log: Compliance and Data Protection . . . 53
36.2.1 PCI DSS (Payment Card Industry Data Security Standard) . . . 53
36.2.2 GDPR (General Data Protection Regulation) . . . 53
36.2.3 HIPAA (Health Insurance Portability and Accountability Act) . . . 53
36.2.4 NEN 7510 (Dutch Healthcare Information Security) . . . 53
36.2.5 SOX (Sarbanes-Oxley Act — Financial) . . . 53
36.3 Designing for Compliance . . . 53
36.3.1 The Principle: Log Metadata, Not Data . . . 53
36.3.2 Audit Logging Pattern . . . 53
36.3.3 Masking in Error Messages . . . 53
36.4 Prometheus Metrics (Safe by Design) . . . 53
37 Quality Assurance and Testing . . . 54
37.1 UTL-X's Layered Testing Strategy . . . 54
37.2 Layer 1: Unit Tests (Kotlin) . . . 54
37.3 Layer 2: Format Tests (Kotlin) . . . 54
37.4 Layer 3: Module Tests (Kotlin) . . . 54
37.5 Layer 4: Integration Tests (Kotlin) . . . 54
37.6 Layer 5: Conformance Suite (Python) . . . 54
37.7 Auto-Capture: Recording Tests from CLI Usage . . . 54

37.8 UTLXe Engine Tests . . . 54
37.9 Test-Driven Transformation Development . . . 54
37.10 Continuous Integration . . . 54
38 Semantic Validation: Beyond Syntax . . . 56
38.1 Syntactical vs Semantic Validation . . . 56
38.1.1 What UTL-X Does Today: Syntactical Validation . . . 56
38.1.2 What UTL-X Does NOT Do: Semantic Validation . . . 56
38.1.3 The Gap . . . 56
38.2 Semantic Validation Languages . . . 56
38.2.1 Schematron (XML world) . . . 56
38.2.2 JSONTron . . . 56
38.2.3 FEEL (Friendly Enough Expression Language) . . . 56
38.2.4 Other Approaches . . . 56
38.3 How Semantic Validation Could Work in UTL-X . . . 56
38.3.1 Option 1: Built-in assertion syntax . . . 56
38.3.2 Option 2: Schematron support for XML output . . . 56
38.3.3 Option 3: FEEL integration for cross-format validation . . . 56
38.3.4 Option 4: Validation DSL (UTL-X native) . . . 56
38.3.5 Recommended Approach . . . 56
39 API Contracts and Data Contracts . . . 57
39.1 What Is an API Contract? . . . 57
39.2 Two Dimensions of Contracts . . . 57
39.2.1 Hierarchical Contracts (API-Centric) . . . 57
39.2.2 Relational Contracts (Data-Centric) . . . 57
39.2.3 Key Difference . . . 57
39.3 Hierarchical API Contract Standards . . . 57
39.3.1 OpenAPI (REST APIs) . . . 57
39.3.2 RAML (REST APIs — MuleSoft) . . . 57
39.3.3 SOAP / WSDL (XML Web Services) . . . 57
39.3.4 AsyncAPI (Event-Driven APIs) . . . 57
39.3.5 gRPC / Protocol Buffers . . . 57
39.4 Abstract vs Concrete API Contracts . . . 57
39.4.1 Abstract Contracts (Schema-Only) . . . 57
39.4.2 Concrete Contracts (Full API Specification) . . . 57
39.4.3 The Relationship . . . 57
39.5 Relational Data Contract Standards . . . 57
39.5.1 Data Contract (datacontract.com) . . . 57
39.5.2 ODCS (Open Data Contract Standard — PayPal) . . . 57
39.5.3 Comparison: datacontract.com vs ODCS . . . 57
39.6 How UTL-X Relates to API and Data Contracts . . . 57
39.6.1 UTL-X as the Transformation Layer . . . 57
39.6.2 Schema Formats in UTL-X . . . 57
39.6.3 Future: Data Contract Integration . . . 57
39.6.4 Future: Contract-Driven Transformation . . . 57
40 Formats Not Yet Covered (and Why) . . . 58
40.1 Formats on the Horizon . . . 58

40.2 TOML (Tom's Obvious Minimal Language) . . . 58
40.3 JSON-LD (JSON for Linked Data) . . . 58
40.4 HL7 FHIR (Fast Healthcare Interoperability Resources) . . . 58
40.5 EDI / EDIFACT (Electronic Data Interchange) . . . 58
40.6 HL7 v2 (Health Level Seven Version 2) . . . 58
40.7 Parquet / Arrow (Columnar Data) . . . 58
40.8 MessagePack / CBOR (Binary JSON) . . . 58
40.9 GraphQL . . . 58
40.10 Apache Thrift . . . 58
40.11 FlatBuffers . . . 58
40.12 Summary: Format Priority Matrix . . . 58
41 Competitive Landscape: Mapping and Integration Tools . . . 59
41.1 Command-Line Transformation Tools . . . 59
41.1.1 jq — The JSON Swiss Knife . . . 59
41.1.2 yq — YAML/JSON/XML/CSV/TOML Processor . . . 59
41.1.3 JSONata — JSON Query and Transformation . . . 59
41.1.4 XSLT — The XML Veteran . . . 59
41.2 Enterprise Integration Platforms (iPaaS) . . . 59
41.2.1 MuleSoft (Salesforce) . . . 59
41.2.2 Tibco BusinessWorks (BWCE) . . . 59
41.2.3 Boomi (formerly Dell Boomi) . . . 59
41.2.4 Workato . . . 59
41.2.5 Informatica / IICS . . . 59
41.3 Hyperscaler Native Tools . . . 59
41.3.1 Azure . . . 59
41.3.2 AWS . . . 59
41.3.3 GCP . . . 59
41.3.4 Hyperscaler Comparison Table . . . 59
41.4 ERP Vendors . . . 59
41.4.1 SAP . . . 59
41.4.2 Oracle . . . 59
41.4.3 Microsoft Dynamics 365 . . . 59
41.4.4 Odoo . . . 59
41.4.5 Summary: ERP + UTL-X Integration Pattern . . . 59
41.5 Open Source Alternatives . . . 59
41.5.1 Apache Camel . . . 59
41.5.2 Smooks . . . 59
41.5.3 Ballerina . . . 59
41.6 Positioning Matrix . . . 59
42 Open-M: The Integration Controller (Teaser) . . . 60
42.1 What Is Open-M? . . . 60
42.2 How UTL-X Fits in Open-M . . . 60
42.3 Why Open-M + UTL-X? . . . 60
42.4 Open-M Architecture (Preview) . . . 60
42.5 MPPM: Message Processing Pipeline Model . . . 60
42.5.1 What Is MPPM? . . . 60

42.5.2 MPPM Stages . . . 60
42.5.3 MPPM Message Envelope . . . 60
42.5.4 How MPPM Maps to UTL-X . . . 60
42.5.5 Why MPPM Matters for UTL-X Users . . . 60
42.6 What This Means for UTL-X Users . . . 60
43 AI and UTL-X: The Future of Automated Transformation . . . 61
43.1 The Current State: AI-Assisted UTL-X Development . . . 61
43.1.1 What Works Today . . . 61
43.1.2 The Problem: Token Cost and Context . . . 61
43.2 The Vision: A UTL-X Fine-Tuned LLM . . . 61
43.2.1 What Fine-Tuning Would Enable . . . 61
43.2.2 Token Savings Calculation . . . 61
43.3 Training Data: What Would Be Needed . . . 61
43.3.1 Available Training Corpus . . . 61
43.3.2 Additional Training Data Needed . . . 61
43.4 Fine-Tuning Approaches . . . 61
43.4.1 Option A: LoRA Fine-Tune on Open Model . . . 61
43.4.2 Option B: Fine-Tune on Frontier Model . . . 61
43.4.3 Option C: RAG (Retrieval-Augmented Generation) — No Fine-Tuning . . . 61
43.4.4 Recommended Path . . . 61
43.5 Investment Estimate . . . 61
43.5.1 Development Costs . . . 61
43.5.2 Ongoing Costs . . . 61
43.5.3 Revenue Potential . . . 61
43.6 What an Investor Would Want to Know . . . 61
43.6.1 The Opportunity . . . 61
43.6.2 The Moat . . . 61
43.6.3 The Ask . . . 61
43.6.4 Why NOW . . . 61
43.7 Ethical Considerations . . . 61
44 Why Kotlin and GraalVM (and Why Not Other Languages) . . . 62
44.1 Why Kotlin Was the Right Choice . . . 62
44.1.1 Kotlin's Strengths for UTL-X . . . 62
44.1.2 Kotlin vs Alternatives Considered . . . 62
44.2 Why GraalVM on Top of Kotlin . . . 62
44.2.1 GraalVM Native Image . . . 62
44.2.2 GraalVM for Cloud (UTLXe) . . . 62
44.3 Why NOT a JavaScript/TypeScript Version . . . 62
44.3.1 The Temptation . . . 62
44.3.2 Why It Would Fail . . . 62
44.4 Why NOT WebAssembly (WASM) . . . 62
44.4.1 The Promise . . . 62
44.4.2 Why It Doesn't Work (Today) . . . 62
44.5 Why Multiple Language Implementations Lead to Deviation . . . 62
44.5.1 The N-Implementation Problem . . . 62
44.5.2 The Wrapper Pattern Avoids Deviation . . . 62

45 Case Studies and Real-World Use Cases ... 64
45.1 Case Study 1: European E-Invoicing (Peppol / UBL) ... 64
45.2 Case Study 2: Healthcare HL7 FHIR Integration ... 64
45.3 Case Study 3: Financial Services — SWIFT ISO 20022 Migration ... 64
45.4 Case Study 4: Retail — Multi-Channel Order Processing ... 64
45.5 Case Study 5: IoT / Manufacturing — Sensor Data Normalization ... 64
45.6 Case Study 6: Government — Tax Filing Integration ... 64
45.7 Azure Marketplace: Getting Started Guide ... 64
45.8 Recipes: Common Transformation Patterns ... 64
46 Grammar Reference ... 66
46.1 UTLX 1.0 Grammar (EBNF) ... 66
46.2 Header Syntax ... 66
46.3 Output Options ... 66
46.4 Expression Syntax ... 66
46.5 Reserved Words ... 66
46.6 Identifier Rules ... 66
47 Appendices ... 67
47.1 Appendix A: Format Conversion Matrix ... 67
47.2 Appendix B: Error Messages Reference ... 67
47.3 Appendix C: Comparison Tables ... 67
47.4 Appendix D: Industry Standards Reference ... 67
47.5 Appendix E: Glossary ... 67
47.6 Appendix F: Version History ... 67
47.7 Appendix G: License ... 67
48 Standard Library Reference ... 69
48.1 How to Read This Reference ... 69
48.2 Core Functions (49) ... 69
48.3 String Functions (83) ... 69
48.4 Array Functions (67) ... 69
48.5 Math Functions (37) ... 69
48.6 Date/Time Functions (68) ... 69
48.7 Type Functions (27) ... 69
48.8 Object Functions (1) ... 69
48.9 Encoding Functions (30) ... 69
48.10 XML Functions (60) ... 69
48.11 Security Functions (16) ... 69
48.12 JSON Functions (6) ... 69
48.13 Binary Functions (47) ... 69
48.14 Utility Functions (27) ... 69
48.15 Financial Functions (16) ... 69
48.16 Geospatial Functions (8) ... 69
48.17 CSV Functions (12) ... 69
48.18 YAML Functions (22) ... 69

1 Preface

1.1 Why This Book

1.2 Who This Book Is For

1.3 How to Read This Book

1.4 Conventions Used

1.5 Online Resources

Part I

Foundation

Understanding the language, the data model, and the philosophy

2 Introduction

2.1 The Data Transformation Problem

2.2 What Is UTL-X?

2.3 The Swiss Army Knife Analogy

2.4 UTL-X vs The Alternatives

2.5 Architecture Overview

2.6 The UTL-X Ecosystem

2.7 A First Example

3 Licensing: Open Source and Commercial

3.1 UTL-X Is Open Source (AGPL-3.0)

UTL-X is licensed under the **GNU Affero General Public License v3.0** (AGPL-3.0). This is one of the strongest open-source licenses, designed to keep software free and open — even when deployed as a cloud service.

3.2 What AGPL-3.0 Means in Practice

3.2.1 You CAN (without paying anything):

3.2.2 The Key AGPL Condition:

3.2.3 You CANNOT (without a commercial license):

3.3 Usage Scenarios and License Requirements

3.4 Important Clarification: Your .utlx Files Are YOURS

3.5 The Commercial License Model

3.5.1 Why a Commercial License Exists

3.5.2 Commercial License Options

3.5.2.1 Azure Marketplace (Managed Application)

3.5.2.2 GCP / AWS Marketplace (Coming)

3.5.2.3 Direct Commercial License

3.5.3 Dual Licensing Summary

3.6 Frequently Asked Questions

3.6.1 "Can I use UTL-X in my company for free?"

3.6.2 "Do I need to open-source my .utlx files?"

3.6.3 "Can I use UTL-X in my SaaS product?"

3.6.4 "What if I deploy UTL-X from Azure Marketplace?"

3.6.5 "Can I redistribute UTL-X?"

3.6.6 "Does AGPL affect my entire application?"

4 Transformation: The Cornerstone of Integration

4.1 Why Transformation Matters

4.2 The Three Pillars of Integration

4.3 Transformation in ESB Architecture

4.4 Transformation in Workflow/BPM

4.5 Why UTLXe Needs Messaging Capabilities

4.6 The Canonical Data Model Pattern

5 Getting Started

5.1 Installation

5.2 Your First Transformation

5.3 The .utlx File Structure

5.4 Identity Mode (The Flip)

5.5 Expression Mode (-e)

5.6 The CLI Reference

5.7 VS Code Extension

5.8 Project Structure

6 The Three Executables: utlx, utlxd, and utlxe

6.1 One Language, Three Runtimes

6.2 utlx — The CLI (Command Line Interface)

6.3 utlxd — The Daemon (IDE Integration)

6.4 utlxe — The Engine (Production Runtime)

6.5 Comparison Table

6.6 Architecture: Shared Core, Different Shells

6.7 When to Use Which

6.8 The Wrapper Pattern

7 The UTL-X IDE

7.1 IDE Layout and Panels

7.2 Design-Time vs Runtime

7.3 Schema-to-Schema Mapping (Design-Time First)

7.4 Scaffolding

7.5 The UDM Tree Browser

7.6 The Function Library

7.7 The %utlx Header Editor

8 Language Fundamentals

8.1 Scalar Types

8.2 Objects

8.3 Arrays

8.4 The $input Variable

8.5 Variable Binding (let)

8.6 Conditional Expressions

8.7 Comments

8.8 Type Annotations (Optional)

9 The Universal Data Model (UDM)

9.1 Why UDM Exists

9.2 UDM Types

9.3 How Formats Map to UDM

9.4 XML-Specific UDM Conventions

9.5 UDM Navigation

9.6 UDM and Type Coercion

10 Schema-to-Schema Mapping

10.1 Why Schema Mapping Matters

10.2 How Traditional Tools Do It (and Why It's Broken)

10.2.1 The XSLT Approach (Tibco BW, SAP CPI, Azure)

10.2.2 The JSON Problem: Why JSON→XML→XSLT→XML→JSON Is Wrong

10.2.3 The UTL-X Approach: Format-Agnostic Schema Mapping

10.3 Tier 1: Instance Data Mapping

10.3.1 Every Data Format Has an Accompanying Schema Format

10.3.2 Instance-to-Instance Mapping Examples

10.4 Tier 2: Schema-to-Schema (Metadata) Mapping

10.4.1 Why Metadata Mapping Is Powerful

10.4.2 All Possible Schema-to-Schema Combinations

10.4.3 Why This Matters in Practice

10.5 JSONATA: JSON Schema to JSON Schema Mapping

10.6 Nested Schemas: Include and Import

10.7 USDL: The Universal Schema Definition Language

11 USDL: Universal Schema Definition Language

11.1 Why USDL?

11.2 USDL Syntax

11.3 USDL Type System

11.4 USDL Constraints

11.5 USDL ↔ Other Schema Formats

11.6 Using USDL in Transformations

11.7 USDL as Documentation

12 Format Support Overview

12.1 Tier 1: Data Formats

12.2 Tier 2: Schema Formats

12.3 Format Detection

12.4 Format Options

12.5 Format Conversion Matrix

Part II

The Language

Mastering expressions, functions, patterns, and advanced features

13 Expressions and Operators

13.1 Arithmetic Operators

13.2 Comparison Operators

13.3 Logical Operators

13.4 String Operators

13.5 Object Construction

13.6 Array Construction

13.7 Pipe Operator (|>)

13.8 Nullish Coalescing and Safe Navigation

14 Functions and Lambdas

14.1 Lambda Expressions

14.2 User-Defined Functions

14.3 Higher-Order Functions

14.4 Function Composition

14.5 Closures

14.6 The Standard Library (Overview)

15 Standard Library

15.1 String Functions (83 functions)

15.2 Array Functions (67 functions)

15.3 Math Functions (37 functions)

15.4 Date/Time Functions (68 functions)

15.5 Type Functions (27 functions)

15.6 Encoding Functions (30 functions)

15.7 Security Functions (16 functions)

15.8 XML Functions (60 functions)

15.9 Other Categories

16 Security Library and Message Security

16.1 The UTL-X Security Library (stdlib-security)

16.2 Should Messages Be Encrypted in Transit?

16.2.1 Transport-Level Encryption (mTLS)

16.2.2 Message-Level Encryption (End-to-End)

16.2.3 Why Encrypt-Transform-Re-encrypt Is a Maintenance Burden

16.3 Digital Signatures and Integrity

16.3.1 JCS (JSON Canonicalization Scheme)

16.3.2 XML Digital Signatures (XMLDSig)

16.3.3 JWS (JSON Web Signature)

16.4 Future Security Functions (Vision)

16.4.1 Encryption

16.4.2 Signing

16.4.3 Key Management Integration

16.5 Large Files: CMIS and Object Storage References

16.5.1 The Problem with Large Payloads

16.5.2 The Claim Check Pattern

16.5.3 CMIS (Content Management Interoperability Services)

16.5.4 Avoid Base64 Embedding

17 Pattern Matching and Control Flow

17.1 Match Expressions

17.2 Try/Catch

17.3 Conditional Patterns

17.4 Real-World Patterns

18 Schema Validation

18.1 Why Validate?

18.2 Supported Schema Formats

18.3 Pre-Validation and Post-Validation

18.4 JSON Schema Validation

18.5 XSD Validation

18.6 Schema-Driven Transformations

19 Pipeline Chaining and Multi-Hop Transformations

19.1 What Is Pipeline Chaining?

19.2 Why Pipelines?

19.3 Single-Hop vs Multi-Hop

19.3.1 Single-Hop (Simple Transformation)

19.3.2 Multi-Hop (Pipeline)

19.4 Multi-Hop with Additional Inputs per Stage

19.4.1 The Problem

19.4.2 The Solution: Additional Inputs per Hop

19.4.3 When Is Multi-Hop with Additional Inputs Handy?

19.4.3.1 1. Data Enrichment Pipelines

19.4.3.2 2. Multi-System Integration (Hub-and-Spoke)

19.4.3.3 3. Validation with External Reference Data

19.4.3.4 4. Template-Based Output Generation

19.4.3.5 5. Configuration-Driven Transformations

19.5 Defining Pipelines

19.5.1 Pipeline via UTLXe HTTP API

19.5.2 Pipeline via CLI

19.6 Multi-Input Transformations (Single Step)

19.7 Pipeline Patterns

19.7.1 Enrichment Pipeline

19.7.2 Validation Sandwich

19.7.3 Fan-Out

19.7.4 Fan-In (Aggregation)

19.7.5 Conditional Routing

19.8 Performance Characteristics

19.9 Status: Multi-Hop Additional Inputs

Part III

Formats Deep Dive

Working with XML, JSON, CSV, YAML, OData, and schema formats

20 XML Transformations

20.1 XML in UTL-X

20.2 Reading XML

20.3 Writing XML

20.4 XML Attributes in JSON/YAML Output

20.5 Common XML Patterns

20.6 XML Encoding

20.7 XML Namespaces Deep Dive

21 XML Attribute Handling: Design Decisions

21.1 The Fundamental Problem

21.2 How UTL-X Handles Attributes Internally (UDM)

21.3 The _text Convention

21.4 The writeAttributes Option

21.5 What Gets Kept vs Dropped (Default)

21.6 Why Drop by Default? (DataWeave-Compatible)

21.7 JSON vs YAML Consistency

21.8 Industry Conventions Reference

21.9 FHIR XML: The value Attribute Convention

22 JSON Transformations

22.1 JSON in UTL-X

22.2 Reading JSON

22.3 Writing JSON

22.4 JSON-Specific Functions

22.5 Common JSON Patterns

22.6 JSON Lines (JSONL)

23 CSV Transformations

23.1 CSV in UTL-X

23.2 CSV Options

23.3 Reading CSV

23.4 Writing CSV

23.5 CSV-Specific Functions

23.6 Common CSV Patterns

23.7 Regional Formats

24 YAML Transformations

24.1 YAML in UTL-X

24.2 Reading YAML

24.3 Writing YAML

24.4 YAML Output Options

24.5 YAML-Specific Functions

24.6 Common YAML Patterns

24.7 YAML ↔ JSON Conversion

25 OData Transformations

25.1 What Is OData?

25.2 OData in UTL-X

25.3 Reading OData

25.4 Writing OData

25.5 OData-Specific Functions

25.6 Common OData Patterns

25.6.1 Dynamics 365 Integration

25.6.2 SharePoint Integration

25.6.3 SAP OData Integration

25.6.4 Power Platform

25.7 OData Schema (EDMX/CSDL)

25.8 OData ↔ Other Formats

26 Schema Formats

26.1 XSD (XML Schema Definition)

26.2 JSON Schema

26.3 Avro Schema

26.4 Protobuf (proto3)

26.5 OSCH (OData Schema / EDMX)

26.6 TSCH (Table Schema)

26.7 USDL (Universal Schema Definition Language)

27 XSD Design Patterns Deep Dive

27.1 The Eight XML Schema Design Patterns

XML Schema (XSD) can be structured in fundamentally different ways. Understanding these patterns is essential for working with enterprise XML — and for understanding why schema-to-schema mapping is challenging.

27.1.1 1. Russian Doll

27.1.2 2. Salami Slice

27.1.3 3. Venetian Blind

27.1.4 4. Garden of Eden

27.1.5 5. Swiss Army Knife (Hybrid)

27.1.6 6-8. Chameleon, Abstract, Adapter

27.2 Pattern Detection in UTL-X

27.3 Pattern Conversion in UTL-X

27.4 XSD Patterns in Cross-Format Mapping

28 Cross-Format Patterns

28.1 The Power of Format Agnosticism

28.2 XML ↔ JSON Conversion

28.3 CSV ↔ JSON Conversion

28.4 XML → CSV (Flattening)

28.5 JSON → XML (Structuring)

28.6 Multi-Format Pipelines

28.7 Data Normalization Patterns

Part IV

Real-World Applications

Enterprise integration, cloud deployment, and production patterns

29 Enterprise Integration

29.1 UTL-X in Integration Architecture

29.2 Use Case: European E-Invoicing (Peppol/UBL)

29.3 Use Case: Healthcare (HL7 FHIR)

29.4 Use Case: Financial Services (SWIFT/ISO 20022)

29.5 Use Case: Retail/E-Commerce

29.6 Message Broker Integration

29.7 Error Handling in Production

30 Engine Lifecycle: Design-Time, Init-Time, Runtime

30.1 The Three Phases

Every UTL-X transformation goes through three distinct phases. Understanding them is key to performance tuning, debugging, and choosing the right execution strategy.

30.1.1 Phase 1: Design-Time

30.1.2 Phase 2: Init-Time (Load / Compile)

30.1.3 Phase 3: Runtime (Execute)

30.2 Phase Diagram

30.3 How Strategies Affect Each Phase

30.4 Worker Threading Model

31 Cloud Deployment

31.1 UTLXe: The Production Engine

31.2 Azure Deployment

31.3 GCP Deployment

31.4 AWS Deployment

31.5 Docker and Kubernetes

31.6 Health and Monitoring

31.7 Hot Reload

32 SDKs and Wrapper Libraries

32.1 The Wrapper Pattern

32.2 Available SDKs

32.2.1 C# / .NET Wrapper (Built)

32.2.2 Go Wrapper (Built)

32.2.3 Python Wrapper (Planned)

32.2.4 Java / Kotlin Wrapper (Direct)

32.2.5 Node.js / TypeScript Wrapper (Planned)

32.3 Alternative: HTTP API (No SDK Needed)

32.4 Alternative: Dapr Sidecar (Cloud-Native)

32.5 SDK Comparison

33 Migration Guides

33.1 Migrating from XSLT

33.2 Migrating from DataWeave

33.3 Migrating from jq

33.4 Migrating from Custom Code

33.5 Migrating from Tibco BW / BWCE

34 Performance and Optimization

34.1 Execution Strategies

34.2 Performance Characteristics

34.3 Memory Optimization

34.4 GraalVM Native Image

34.5 Benchmarking

34.6 Production Tuning

35 Message Parsing, Memory, and Size Boundaries

35.1 How UTL-X Parses Messages

35.1.1 DOM-Style Parsing (What UTL-X Uses)

35.1.2 SAX-Style Parsing (Not Used by UTL-X)

35.1.3 StAX-Style Parsing (Not Used by UTL-X)

35.1.4 Summary: Parsing Approaches

35.2 Memory Expansion Factor

35.2.1 The Problem: Why 100KB of XML Becomes 5-10MB in Memory

35.2.2 Expansion Factors by Format

35.2.3 Real-World Examples

35.3 Practical Message Size Boundaries

35.3.1 Sweet Spot: 1 KB - 1 MB

35.3.2 Feasible: 1 MB - 10 MB

35.3.3 Not Recommended: 10 MB - 50 MB

35.3.4 Not Feasible: 50 MB+

35.3.5 Size Boundary Summary

35.4 Memory Sizing Guide for Production

35.4.1 Formula

35.4.2 The 75% Rule

35.5 Future: Streaming Support?

35.5.1 Could UTL-X Support Streaming?

35.5.2 Split-Before-Transform Pattern

35.6 Comparison: UTL-X vs Streaming Tools

36 Logging, Observability, and Compliance

36.1 Logging in UTL-X

36.1.1 What UTL-X Logs

36.1.2 Log Levels

36.1.3 Configuring Logging

36.2 When NOT to Log: Compliance and Data Protection

36.2.1 PCI DSS (Payment Card Industry Data Security Standard)

36.2.2 GDPR (General Data Protection Regulation)

36.2.3 HIPAA (Health Insurance Portability and Accountability Act)

36.2.4 NEN 7510 (Dutch Healthcare Information Security)

36.2.5 SOX (Sarbanes-Oxley Act — Financial)

36.3 Designing for Compliance

36.3.1 The Principle: Log Metadata, Not Data

36.3.2 Audit Logging Pattern

36.3.3 Masking in Error Messages

36.4 Prometheus Metrics (Safe by Design)

37 Quality Assurance and Testing

37.1 UTL-X's Layered Testing Strategy

37.2 Layer 1: Unit Tests (Kotlin)

37.3 Layer 2: Format Tests (Kotlin)

37.4 Layer 3: Module Tests (Kotlin)

37.5 Layer 4: Integration Tests (Kotlin)

37.6 Layer 5: Conformance Suite (Python)

37.7 Auto-Capture: Recording Tests from CLI Usage

37.8 UTLXe Engine Tests

37.9 Test-Driven Transformation Development

37.10 Continuous Integration

Part V

Future Outlook

Semantic validation, API contracts, and the road ahead

38 Semantic Validation: Beyond Syntax

38.1 Syntactical vs Semantic Validation

38.1.1 What UTL-X Does Today: Syntactical Validation

38.1.2 What UTL-X Does NOT Do: Semantic Validation

38.1.3 The Gap

38.2 Semantic Validation Languages

38.2.1 Schematron (XML world)

38.2.2 JSONTron

38.2.3 FEEL (Friendly Enough Expression Language)

38.2.4 Other Approaches

38.3 How Semantic Validation Could Work in UTL-X

38.3.1 Option 1: Built-in assertion syntax

38.3.2 Option 2: Schematron support for XML output

38.3.3 Option 3: FEEL integration for cross-format validation

38.3.4 Option 4: Validation DSL (UTL-X native)

38.3.5 Recommended Approach

39 API Contracts and Data Contracts

39.1 What Is an API Contract?

39.2 Two Dimensions of Contracts

39.2.1 Hierarchical Contracts (API-Centric)

39.2.2 Relational Contracts (Data-Centric)

39.2.3 Key Difference

39.3 Hierarchical API Contract Standards

39.3.1 OpenAPI (REST APIs)

39.3.2 RAML (REST APIs — MuleSoft)

39.3.3 SOAP / WSDL (XML Web Services)

39.3.4 AsyncAPI (Event-Driven APIs)

39.3.5 gRPC / Protocol Buffers

39.4 Abstract vs Concrete API Contracts

39.4.1 Abstract Contracts (Schema-Only)

39.4.2 Concrete Contracts (Full API Specification)

39.4.3 The Relationship

39.5 Relational Data Contract Standards

39.5.1 Data Contract (datacontract.com)

39.5.2 ODCS (Open Data Contract Standard — PayPal)

39.5.3 Comparison: datacontract.com vs ODCS

39.6 How UTL-X Relates to API and Data Contracts

39.6.1 UTL-X as the Transformation Layer

39.6.2 Schema Formats in UTL-X

39.6.3 Future: Data Contract Integration

39.6.4 Future: Contract-Driven Transformation

40 Formats Not Yet Covered (and Why)

40.1 Formats on the Horizon

UTL-X currently supports 5 data formats (Tier 1) and 6 schema formats (Tier 2). Several important formats are NOT yet supported. This chapter explains each, why they matter, and the path to support.

40.2 TOML (Tom's Obvious Minimal Language)

40.3 JSON-LD (JSON for Linked Data)

40.4 HL7 FHIR (Fast Healthcare Interoperability Resources)

40.5 EDI / EDIFACT (Electronic Data Interchange)

40.6 HL7 v2 (Health Level Seven Version 2)

40.7 Parquet / Arrow (Columnar Data)

40.8 MessagePack / CBOR (Binary JSON)

40.9 GraphQL

40.10 Apache Thrift

40.11 FlatBuffers

40.12 Summary: Format Priority Matrix

41 Competitive Landscape: Mapping and Integration Tools

41.1 Command-Line Transformation Tools

41.1.1 jq — The JSON Swiss Knife

41.1.2 yq — YAML/JSON/XML/CSV/TOML Processor

41.1.3 JSONata — JSON Query and Transformation

41.1.4 XSLT — The XML Veteran

41.2 Enterprise Integration Platforms (iPaaS)

41.2.1 MuleSoft (Salesforce)

41.2.2 Tibco BusinessWorks (BWCE)

41.2.3 Boomi (formerly Dell Boomi)

41.2.4 Workato

41.2.5 Informatica / IICS

41.3 Hyperscaler Native Tools

41.3.1 Azure

41.3.2 AWS

41.3.3 GCP

41.3.4 Hyperscaler Comparison Table

41.4 ERP Vendors

41.4.1 SAP

41.4.2 Oracle

41.4.3 Microsoft Dynamics 365

41.4.4 Odoo

41.4.5 Summary: ERP + UTL-X Integration Pattern

41.5 Open Source Alternatives

41.5.1 Apache Camel

41.5.2 Smooks

41.5.3 Ballerina

41.6 Positioning Matrix

42 Open-M: The Integration Controller (Teaser)

42.1 What Is Open-M?

42.2 How UTL-X Fits in Open-M

42.3 Why Open-M + UTL-X?

42.4 Open-M Architecture (Preview)

42.5 MPPM: Message Processing Pipeline Model

42.5.1 What Is MPPM?

42.5.2 MPPM Stages

42.5.3 MPPM Message Envelope

42.5.4 How MPPM Maps to UTL-X

42.5.5 Why MPPM Matters for UTL-X Users

42.6 What This Means for UTL-X Users

43 AI and UTL-X: The Future of Automated Transformation

43.1 The Current State: AI-Assisted UTL-X Development

43.1.1 What Works Today

43.1.2 The Problem: Token Cost and Context

43.2 The Vision: A UTL-X Fine-Tuned LLM

43.2.1 What Fine-Tuning Would Enable

43.2.2 Token Savings Calculation

43.3 Training Data: What Would Be Needed

43.3.1 Available Training Corpus

43.3.2 Additional Training Data Needed

43.4 Fine-Tuning Approaches

43.4.1 Option A: LoRA Fine-Tune on Open Model

43.4.2 Option B: Fine-Tune on Frontier Model

43.4.3 Option C: RAG (Retrieval-Augmented Generation) — No Fine-Tuning

43.4.4 Recommended Path

43.5 Investment Estimate

43.5.1 Development Costs

43.5.2 Ongoing Costs

43.5.3 Revenue Potential

43.6 What an Investor Would Want to Know

43.6.1 The Opportunity

43.6.2 The Moat

43.6.3 The Ask

43.6.4 Why NOW

43.7 Ethical Considerations

44 Why Kotlin and GraalVM (and Why Not Other Languages)

44.1 Why Kotlin Was the Right Choice

44.1.1 Kotlin's Strengths for UTL-X

44.1.2 Kotlin vs Alternatives Considered

44.2 Why GraalVM on Top of Kotlin

44.2.1 GraalVM Native Image

44.2.2 GraalVM for Cloud (UTLXe)

44.3 Why NOT a JavaScript/TypeScript Version

44.3.1 The Temptation

44.3.2 Why It Would Fail

44.4 Why NOT WebAssembly (WASM)

44.4.1 The Promise

44.4.2 Why It Doesn't Work (Today)

44.5 Why Multiple Language Implementations Lead to Deviation

44.5.1 The N-Implementation Problem

44.5.2 The Wrapper Pattern Avoids Deviation

Part VI

Case Studies

Real-world use cases, step-by-step walkthroughs, and recipes

45 Case Studies and Real-World Use Cases

45.1 Case Study 1: European E-Invoicing (Peppol / UBL)

45.2 Case Study 2: Healthcare HL7 FHIR Integration

45.3 Case Study 3: Financial Services — SWIFT ISO 20022 Migration

45.4 Case Study 4: Retail — Multi-Channel Order Processing

45.5 Case Study 5: IoT / Manufacturing — Sensor Data Normalization

45.6 Case Study 6: Government — Tax Filing Integration

45.7 Azure Marketplace: Getting Started Guide

45.8 Recipes: Common Transformation Patterns

Part VII

Reference

Grammar, appendices, and general reference

46 Grammar Reference

46.1 UTLX 1.0 Grammar (EBNF)

46.2 Header Syntax

46.3 Output Options

46.4 Expression Syntax

46.5 Reserved Words

46.6 Identifier Rules

47 Appendices

47.1 Appendix A: Format Conversion Matrix

47.2 Appendix B: Error Messages Reference

47.3 Appendix C: Comparison Tables

47.4 Appendix D: Industry Standards Reference

47.5 Appendix E: Glossary

47.6 Appendix F: Version History

47.7 Appendix G: License

Part VIII

Standard Library Encyclopedia

Complete reference for all 652 functions — organized by category with examples

48 Standard Library Reference

48.1 How to Read This Reference

48.2 Core Functions (49)

48.3 String Functions (83)

48.4 Array Functions (67)

48.5 Math Functions (37)

48.6 Date/Time Functions (68)

48.7 Type Functions (27)

48.8 Object Functions (1)

48.9 Encoding Functions (30)

48.10 XML Functions (60)

48.11 Security Functions (16)

48.12 JSON Functions (6)

48.13 Binary Functions (47)

48.14 Utility Functions (27)

48.15 Financial Functions (16)

48.16 Geospatial Functions (8)

48.17 CSV Functions (12)

48.18 YAML Functions (22)